

# Mini Assessment: API & JSON Handling

Name: Sorna parvathi. M

Register Number: 99230041128

Department: CSE

---

## Question 1: If an API returns 429, what should your program do?

HTTP status code **429 (Too Many Requests)** indicates that the client has exceeded the API's designated rate limit within a specified timeframe. When receiving a 429 status code, the program should avoid immediate repeated requests or unhandled crashes and follow these best practices:

- **Check for the 'Retry-After' Header:** Inspect the response headers for a `Retry-After` field. If provided, pause execution (e.g., `time.sleep()`) for the exact number of seconds or until the reset timestamp specified.
- **Implement Exponential Backoff with Jitter:** If no header is provided, double the wait duration on each retry (e.g., 1s, 2s, 4s, 8s). Adding small randomized jitter prevents concurrent clients from synchronizing retries and overwhelming the server.
- **Enforce a Maximum Retry Ceiling:** Set a strict limit (typically 3 to 5 retries). Once exceeded, terminate gracefully instead of looping indefinitely.
- **Log and Fallback Gracefully:** Log rate-limiting events for observability and provide a safe fallback (such as cached results or a clear user-facing message).

```
import time
import requests

def fetch_data(url, retries=3, delay=2):
    for attempt in range(retries):
        response = requests.get(url)

        if response.status_code == 200:
            return response.json()
        elif response.status_code == 429:
            # Check Retry-After header or use exponential backoff
            retry_after = response.headers.get("Retry-After")
            wait_time = int(retry_after) if retry_after else delay * (2 ** attempt)
            print(f"Rate limited (429). Retrying in {wait_time}s...")
            time.sleep(wait_time)
        else:
            response.raise_for_status()

    raise Exception("Request failed: API rate limit exceeded after maximum retries.")
```

## Question 2: If `response.json()` fails, what should your program do?

A failure during `response.json()` (such as a `JSONDecodeError`) indicates that the server's response body is malformed, empty, or not valid JSON. This often occurs when web servers or proxies return an HTML error page (e.g., 500 Internal Server Error, 502/504 Bad Gateway) or plain text. The program should handle this defensively:

- **Catch Specific Exceptions:** Enclose the call in a `try...except` (`requests.exceptions.JSONDecodeError`, `ValueError`) block.
- **Validate Status Code and Content-Type:** Verify if `response.status_code` indicates an error and check whether `Content-Type` is `application/json` before parsing.
- **Log Raw Response Body:** Log a truncated snippet of `response.text[:200]` to diagnose whether HTML, gateway errors, or empty strings were received.
- **Provide a Graceful Fallback:** Return a safe default value (such as `None` or `{}`) or raise a custom domain error rather than unhandled termination.

```
import requests

def safe_get_json(url):
    response = requests.get(url)

    # Check if the server returned an error status
    if not response.ok:
        print(f"Server returned HTTP status: {response.status_code}")

    try:
        return response.json()
    except requests.exceptions.JSONDecodeError:
        print(f"Failed to parse JSON. Raw response preview: {response.text[:150]}")
        return None # Or raise a descriptive custom exception
```

### Question 3: If a JSON key is missing, why might .get() be safer than []?

In Python dictionaries, accessing keys via `.get()` is significantly safer than using bracket notation `[]` when consuming API responses:

- **Avoids Unhandled KeyError:** Indexing with `data["key"]` raises an immediate `KeyError` if the key is missing, causing script termination unless manually wrapped in try-catch blocks. Conversely, `data.get("key")` returns `None` safely.
- **Provides Built-in Fallbacks:** The `.get()` method accepts a second argument as a default value (e.g., `data.get("role", "Student")`), simplifying code without requiring manual key checks.
- **Resilience to Dynamic Schemas:** Web APIs frequently evolve, omit null properties, or return optional fields. Using `.get()` prevents minor payload variations from breaking production code.

| Aspect               | Bracket Notation: <code>data['key']</code>                      | Method: <code>data.get('key')</code>                            |
|----------------------|-----------------------------------------------------------------|-----------------------------------------------------------------|
| Missing Key Behavior | Throws <code>KeyError</code> ; terminates program if unhandled. | Returns <code>None</code> ; program execution continues safely. |
| Default Fallback     | Requires manual <code>if key in dict</code> validation.         | Built-in parameter: <code>data.get('key', default)</code> .     |
| Best Use Case        | Mandatory schema keys that must exist ("fail-fast").            | Optional, nullable, or evolving API response fields.            |

```
data = {"id": 101, "name": "Sorna"}

# 1. Bracket notation (Unsafe for missing keys):
# email = data["email"] # Raises KeyError: 'email'

# 2. .get() method (Safe and defensive):
email = data.get("email")           # Returns None (no crash)
role = data.get("role", "Student")  # Returns 'Student' (fallback)

print(f"Name: {data['name']}, Email: {email}, Role: {role}")
```